National Institute of Business Management

School of Computing and Engineering

Coursework | Assessment Announcement Sheet

iOS Application Development — BSC (Hons) Computing — Batch BSCCOMP25.1P

Course Name	BSC (Hons) Computing
Module Name	iOS Application Development
Batch	BSCCOMP25.1P
Learning Outcomes Covered (Mention according to the Module Descriptor)	LO1 — Critically evaluate the platform components and their uses across multiple devices and formats LO2 — Design and evaluate software suitable for mobile architectures LO3 — Develop and evaluate apps that can interact with external APIs and devices LO4 — Develop and evaluate apps that can communicate with sensors built into the phone hardware
Assessment | CW No	**Final Coursework — 100% Weightage (Part 1 of 1)**
Assessment Mode	**Individual**
Assessment Type	Practical Test | Report | Software | Presentation | VIVA
Hand in Date | Time	*To be announced on LMS*
Hand out Date | Time	*To be announced on LMS*
Submission Details (Format and Location)	Document via LMS Zip file of code and any resources needed to run the application in a development environment GitHub URL for source code hosted in a private repository — grant the lecturer access and submit the commit hash of the final work Screen recording of the application submitted, with the student describing the app's features
Plagiarism Criteria	As per NIBM academic integrity policy

Assessment | Coursework Description

Objective

Students are required to design, propose, and implement a fully functional iOS application. This coursework is designed to assess the student's understanding and practical skills in iOS development, from idea conception to final implementation, and is assessed entirely on an individual basis across six weighted components.

Note: Plagiarism will not be tolerated. All work submitted must be the student's own, and any external resources used must be properly cited.

Disclaimer on the Use of Large Language Models (LLMs)

Prohibited Use of LLMs for Code Generation: Students are strictly prohibited from using Large Language Models (LLMs) such as ChatGPT, Gemini, Claude, or similar AI tools for generating executable code, code snippets, or implementation logic. The core programming work must be entirely the student's own.

Permitted Use of LLMs: LLMs may be used for non-code-related tasks such as brainstorming ideas, outlining project documentation, or generating text for the final report. The use of LLMs in these areas must be clearly documented.

- **Documentation of LLM Use:** If LLMs are used for any permitted tasks, students must explicitly state this in their final report, detailing the specific tasks for which the LLM was used and how the outputs were utilised.
- **Proof of Understanding:** Students must demonstrate a clear and thorough understanding of all work related to their project, especially areas where LLMs were utilised for non-code tasks. This will be assessed during the viva.

Academic Integrity: Any violation of this policy, including the use of LLMs for code generation or failure to properly document the use of LLMs for other tasks, will be considered a breach of academic integrity and may result in significant penalties, including a reduction in marks or referral for academic misconduct.

Assessment Structure & Weightage

This coursework contributes 100% to the overall module grade and is assessed individually across the six components below. Full grading criteria for each component are provided in the accompanying Marking Rubric document.

No.	Component	Weight
1	Prototype & MVP Approval	10%
2	Implementation of MVP & Presentation	20%
3	Implementation of Advanced iOS SDK	20%
4	Unit Tests, Accessibility & Documentation	15%
5	Above and Beyond	25%
6	In-Class Tutorial 4 (Marked)	10%
	Total	**100%**

To pass this module, an overall average of 40% or above must be achieved across all six components combined.

Learning Outcomes Mapping

Each module learning outcome maps to specific components. Students are also required to include a short Learning Outcomes Reflection in their final report (see Component 4) explaining how their project evidences each outcome.

LO	Outcome	Primary Evidence
LO1	Platform components across devices	Component 5 (Above and Beyond) — Multi-Device Design & Adaptation layer, plus the Learning Outcomes Reflection in the final report (Component 4).
LO2	Mobile architecture design	Components 1, 2 and 5 — proposal, MVP implementation, and architecture/code quality.
LO3	External APIs and devices	Component 2 — mandatory external API/backend integration for every student, plus Component 3 for a more advanced implementation.
LO4	Device sensors	Component 6 — compulsory Core Location in Tutorial 4, plus sensor-driven options in Component 3.

Submission Points

Although the coursework is graded across six components, there are four physical touch-points. Components 2, 3, 4 and 5 are all graded from the same single Final App Submission — there is no separate hand-in per component within that group. The viva is a separate, live event.

Submission	Covers	What Is Submitted
1. Proposal & Prototype	Component 1	Proposal document and high-fidelity prototype file(s), via LMS.
2. Final App Submission	Components 2, 3, 4 & 5	One single submission: the completed app, source code zip, GitHub repository URL with the final commit hash, screen recording, and final report. This one commit is what is graded across all four components.
3. Tutorial 4 Submission	Component 6	Completed Xcode project and Git commit history from the in-class lab session.
4. Viva	Component 2	A separate, live, structured Q&A — distinct from the narrated screen recording (see "Viva Structure" under Component 2).

Component 1 — Prototype & MVP Approval (10% of Coursework)

Task

Students must brainstorm and propose an idea for an iOS application that solves a real problem they have personally faced or observed. If a student chooses to emulate an existing application, they must add significant, justifiable features that meaningfully enhance the user's experience or functionality.

Proposal Requirements

- A concise description (1–2 paragraphs) of the app idea, including the target audience, core functionality, and unique selling points (USP).
- Feature Descriptions: a detailed description of the app's features, focusing on how the app will function and the problem it solves.
- UI/UX Prototype: a high-fidelity prototype (e.g. Figma) reflecting iOS design standards, showcasing the expected look, feel, key interactions, and transitions. This is the complete design deliverable — low-fidelity wireframes are not required.
- Tools and Services Justification: an outline of the tools, libraries, frameworks, or third-party services planned, with justification for each choice.

Approval Process

Proposals will be reviewed by the lecturer, who will provide feedback and outline a Minimum Viable Product (MVP) that must be completed as the baseline for assessment. Students may need to revise and resubmit their proposal based on feedback. Only approved proposals may proceed to the implementation phase.

Submission

- Proposal document and prototype file(s) submitted via LMS.

Grading: *See "Component 1 Rubric" in the accompanying Marking Rubric document for full grading criteria and subcomponent breakdown.*

Component 2 — Implementation of MVP & Presentation (20% of Coursework)

Task

Students must implement the core features described in their approved proposal. The app must be functional, modular, well-commented, well-named, and easy to read, closely following the planned UX flow and MVP scope agreed during Component 1.

Mandatory MVP Features — Choose 2 of 3

As part of the core MVP, every student must implement two of the following three platform features. These two chosen features are assessed as part of the Component 2 MVP baseline.

- Push Notifications — implement notifications to keep users informed of updates or important information related to the app.
- Core Data for Local Storage — use Core Data to manage local data storage and ensure data persistence across app launches.
- User Authentication with Face ID / Touch ID — implement biometric authentication for secure user login.

Note: None of these three features — Push Notifications, Core Data, and Face ID/Touch ID — may be used as the Component 3 advanced feature, whether or not they were chosen for the MVP. Component 3 must be built on a different feature entirely.

Mandatory External API / Backend Integration (LO3)

In addition to the two chosen platform features above, every student's app must integrate at least one genuine external API or backend service — for example, a public REST API, Firebase (Firestore, Realtime Database, or Auth), or a custom serverless function (e.g. AWS Lambda, Vercel or Cloudflare Functions) — used to fetch, sync, or persist data that is central to the app's purpose. This is a baseline requirement for every student, separate from the advanced iOS SDK feature required in Component 3.

Presentation & Demonstration

Students must present and demonstrate their MVP, explaining the app's structure, functionality, and key design decisions. This will take the form of an in-person or recorded walkthrough where students must be able to defend their implementation choices and respond to follow-up questions.

Viva Structure

The viva is a separate, live event from the narrated screen recording submitted with the Final App Submission below — it is not the same artifact. It is structured to verify the student's work product and assess their understanding and skill, around question categories such as (but not limited to):

- Work Product — verifying the code is genuinely the student's own.
- Understanding — probing comprehension of the student's own design decisions.
- Skill — assessing technical competency and the ability to problem-solve live.

Final App Submission (Covers Components 2, 3, 4 & 5)

Components 2 through 5 are all graded from one single final submission — there is no separate hand-in per component. The Final App Submission must include:

- The completed application built in Xcode, matching (or improving on) the approved proposal.
- Complete source code, including a zip file of the code and any resources needed to run the application.
- GitHub repository URL (private, lecturer granted access) with the commit hash of the final submission — this single commit is what is graded across Components 2, 3, 4 and 5.
- Screen recording of the application, narrated by the student, describing its features.
- Final report (see Component 4).

Deliverables

- Functional MVP with mandatory external integration, submitted as part of the Final App Submission above.
- A separate live viva (see "Viva Structure").

Grading: *See "Component 2 Rubric" in the accompanying Marking Rubric document for full grading criteria and subcomponent breakdown.*

Component 3 — Implementation of Advanced iOS SDK (20% of Coursework)

Task

In addition to the two mandatory MVP features and the mandatory external integration completed under Component 2, students must integrate at least one further advanced iOS SDK feature that goes beyond standard UI components and meaningfully enhances the application. Examples include (but are not limited to):

- MapKit and Core Location (wayfinding, geofencing, location-aware features)
- SiriKit / App Intents (Shortcuts integration)
- Core ML (on-device machine learning, using a pre-trained or converted model)
- Vision (on-device image analysis — e.g. text or object detection)
- Natural Language (on-device text analysis — e.g. sentiment, entity recognition)
- ARKit
- HealthKit or Core Motion (sensor/health-driven features)
- Core Bluetooth (communicating with nearby BLE devices)
- WidgetKit (Home Screen widgets reflecting live app data)
- CloudKit (iCloud sync across a user's devices)
- Speech framework (on-device speech-to-text)

Note: Push Notifications, Core Data, and Face ID/Touch ID are reserved for Component 2 and cannot be used to satisfy this component, regardless of which two were chosen for the MVP. Similarly, a basic/baseline external API call already counted under Component 2 cannot be re-used here — only a demonstrably deeper implementation qualifies.

Comprehensive vs Token Usage

Importing a framework and calling a single basic method is not sufficient — the feature must be comprehensively integrated and meaningfully solve part of the app's problem. Token usage will not earn any marks toward this component, regardless of which SDK is used. For example:

- Token usage (no marks): a single hardcoded pin dropped on a MapKit view with no underlying data or interactivity.
- Comprehensive usage (required standard): live pins driven by real data, with geofencing that triggers alerts as the user enters or leaves a defined region.

On-Device Machine Learning & AI

Any machine learning, computer vision, or AI feature must run entirely on-device, using frameworks such as Core ML, Vision, Natural Language, or Create ML. Pre-trained models may be used, converted to Core ML format.

Making network calls to an external AI/LLM API (e.g. OpenAI, Gemini, or similar) does not satisfy this requirement and will not earn any marks toward Component 3. A chatbot or conversational feature powered by a remote/cloud LLM rather than on-device inference will similarly not earn marks.

Expectations

The chosen advanced feature(s) must be relevant to the app's problem domain, deeply integrated into the core app logic (not a superficial add-on), and demonstrate technical complexity beyond typical CRUD or navigation functionality. Multiple well-integrated advanced features are required for full marks.

Deliverables

- Working, comprehensively integrated implementation of the advanced feature(s), submitted as part of the single Final App Submission.
- Explanation of the feature's implementation and rationale in the final report and viva.

Grading: *See "Component 3 Rubric" in the accompanying Marking Rubric document for full grading criteria and subcomponent breakdown.*

Component 4 — Unit Tests, Accessibility & Documentation (15% of Coursework)

Unit Tests

The project must include unit tests for key components where applicable, using the Swift Testing framework (not XCTest), to ensure the app functions as expected. Tests should be well-documented and demonstrate comprehensive coverage of core logic and edge cases.

Accessibility

The app must be designed with accessibility in mind, ensuring it is usable by people with varying abilities. This includes support for VoiceOver, Dynamic Type, and other relevant iOS accessibility features.

Documentation

Code should be modular, well-commented, and follow standard naming conventions. The final report must include an overview of the app, a discussion of the development process and challenges faced, a description of the advanced iOS feature(s) implemented, and a reflection on the module's learning outcomes.

Learning Outcomes Reflection

The final report must include a short, dedicated section reflecting on how the project evidences each of the four module learning outcomes (LO1–LO4). For LO1 specifically, justify the app's design choices across device types, drawing on the Component 5 multi-device design work.

Deliverables

- Unit test suite, accessibility features, and the final report (including the Learning Outcomes Reflection) — all submitted as part of the single Final App Submission (see "Submission Points" on an earlier page).

Grading: *See "Component 4 Rubric" in the accompanying Marking Rubric document for full grading criteria and subcomponent breakdown.*

Component 5 — Above and Beyond (25% of Coursework)

Task

This component rewards additional effort, creativity, and technical sophistication shown across the project as a whole, beyond the baseline MVP and the mandatory advanced feature. This is assessed holistically once the rest of the project has been reviewed.

Examples of Above and Beyond Work

- Multi-Device Design & Adaptation (LO1) — designing and implementing the app across at least two iOS device types (e.g. iPhone and iPad), with genuine layout/navigation adaptation, and justifying those platform-specific design choices in the final report.
- Multiple well-integrated advanced iOS features rather than the single mandatory one.
- Sophisticated app architecture (e.g. clean MVVM separation, dependency injection, protocol-oriented design).
- Performance-aware implementation, thoughtful error handling, or offline support.
- Genuinely novel functionality that goes beyond the scope discussed at proposal approval.
- Polish and attention to detail — thoughtful micro-interactions, animations, and well-crafted empty/error states.

Assessment Basis

Marks are awarded across five dimensions: creativity and innovation, technical sophistication, multi-device design and adaptation, code quality and architecture, and overall polish/user experience. See the Component 5 Rubric for the full breakdown.

Grading: *See "Component 5 Rubric" in the accompanying Marking Rubric document for full grading criteria and subcomponent breakdown.*

Component 6 — In-Class Tutorial 4 (Marked) — "The Real App" (10% of Coursework)

Context

This component is based on Tutorial 4 — "The Real App" — due the Saturday following its release and completed primarily during the scheduled lab session, as the final task of the four-week tutorial programme. It builds on the same evolving app used in Tutorials 1–3 (Tap Frenzy, Light It Up, Quiz Rush), wrapping all three game modes into a single, structured, polished app shell with real platform features.

Three modes on their own is a portfolio. Three modes inside a structured, polished app is a project. Wrapping everything into a proper app shell with real platform features is what turns it into "The Real App."

Suggested Structure (Refactor First)

Refactoring the existing tutorial code into a clear, organised structure is strongly recommended before new features are added. A suggested structure (compact form) is shown below — students may adapt this to their own project as needed:

```
PlayHubApp/
   App/  PlayHubApp.swift
   Models/  GameMode.swift, GameSession.swift, TriviaQuestion.swift
   ViewModels/  TapFrenzyVM.swift, LightItUpVM.swift, QuizRushVM.swift, StatsVM.swift
   Services/  TriviaAPI.swift, NotificationService.swift, LocationService.swift
   Views/
      Tabs/  HomeTab.swift, StatsTab.swift, MapTab.swift, SettingsTab.swift
      Games/  TapFrenzyView.swift, LightItUpView.swift, QuizRushView.swift
      Shared/  ResultView.swift, ScoreBadge.swift
```

GameSession Model

A single shared model records every completed game: id (UUID), mode (GameMode), score (Int), timestamp (Date), latitude (Double), longitude (Double). Every completed game appends one GameSession, persisted as an encoded JSON array in UserDefaults.

What to Build

- Tab Bar Shell — a TabView with 4 tabs (Home, Stats, Map, Settings) using SF Symbols for icons; each tab is its own NavigationStack.
- Stats Screen — reads all recorded GameSessions and shows totals, personal bests, a recent-games list, and a bar chart per game mode using the Charts framework (one BarMark per session).
- Map of Games — MapKit + Core Location; each completed session records the player's current location, and the Map tab displays a pin per session (tap a pin to see that session's score).
- Daily Challenge — UserNotifications; the Settings tab lets the player pick a time, and a daily local notification is scheduled at that time.
- Share Your Score — a ShareLink on the Result screen generating a formatted share string (e.g. "I just scored 47 on Quiz Rush — beat that").
- Settings — a notifications toggle, the daily challenge time picker, and a reset-all-stats button with a confirmation dialog.

Deliverables

- Completed Xcode project implementing all Tutorial 4 requirements, submitted via the module Git repository.
- Git commit history showing incremental progress during the session.
- README file (architecture overview, features list, known limitations, reflection).
- A brief in-lab walkthrough/check with the lecturer or lab instructor confirming completion and understanding.

Due the Saturday following release — this is a final marked submission; no late resubmission unless agreed with the lecturer. Full task instructions, the suggested approach, and the submission checklist are provided in the Tutorial 4 brief on the LMS.

***Grading:** See "Component 6 Rubric" in the accompanying Marking Rubric document for full grading criteria and subcomponent breakdown.*

Marking Scheme Summary

This coursework is composed of six individually assessed components, weighted as follows. Full subcomponent-level grading criteria for each component are provided in the accompanying Marking Rubric document.

No.	Component	Weight	Mandatory
1	Prototype & MVP Approval	10%	Yes
2	Implementation of MVP & Presentation	20%	Yes
3	Implementation of Advanced iOS SDK	20%	Yes
4	Unit Tests, Accessibility & Documentation	15%	No
5	Above and Beyond	25%	No
6	In-Class Tutorial 4 (Marked)	10%	No
	Total	**100%**	

Mandatory Components

Components 1, 2 and 3 are mandatory: a zero score in any of these three components will result in failure of the module as a whole, regardless of performance in the remaining components.

Pass Requirement

To pass this module, an overall average of 40% or above must be achieved across all six components combined.

Plagiarism

Plagiarism will not be tolerated. All work submitted must be the student's own, and any external resources used must be properly cited, in line with NIBM's academic integrity policy.